

Astro + Cloudflare R2: Auto-Rebuild Architecture

Generated with GitHub Copilot

2026-10-01

Astro + Cloudflare R2: Automated Rebuild Architecture

Goal

Store all media files, JSON data, SVGs, and videos in a Cloudflare R2 bucket. When content is updated through the Cloudflare/R2 API, automatically trigger an Astro static site rebuild so the published site reflects the new content.

Verdict: Yes, this is possible and is a standard JAMstack “rebuild on content change” pattern.

Architecture Diagram



1. Storage Layer: Cloudflare R2

- Create one or more R2 buckets, e.g. `site-assets`, organized by prefix:
 - `media/`, `data/`, `videos/`, `icons/`
- Manage objects via:
 - The **S3-compatible API** (works with AWS SDK v3, `boto3`, `aws-cli --endpoint-url`), or
 - The **native R2 API / Wrangler CLI** (`wrangler r2 object put/get/delete`)
- Expose content for the Astro build in one of two ways:

- **Public bucket / custom domain:** simplest, lets Astro `fetch()` assets over HTTP at build time.
- **Private bucket + R2 binding:** more secure, accessed only from a Worker or from the build process using S3 SDK credentials stored as secrets.

2. Change Detection: Triggering a Rebuild

R2 has no direct “notify Cloudflare Pages” feature, so bridge it with one of:

Option A — Event Notifications + Queue + Worker (recommended for decoupled updates)

1. Enable **R2 Event Notifications** on the bucket for `PutObject` / `DeleteObject`.
2. Notifications are delivered to a **Cloudflare Queue**.
3. A **Worker** consumes queue messages and calls the Pages **Deploy Hook URL**.
4. This works even if updates come from third parties or multiple scripts, since the trigger is bucket-driven, not caller-driven.

Option B — Direct call from your update script (simpler)

1. Your script/API call that uploads/deletes objects in R2 also performs a second `fetch()` call to the **Deploy Hook URL** right after the R2 operation succeeds.
2. No Queue/Worker needed, but every caller must remember to trigger the hook.

3. Build Layer: Astro Reading from R2

During `astro build`:

- **JSON data:** fetch via `getStaticPaths()` or `frontmatter`, e.g. `const res = await fetch('https://assets.example.com/...')`
- **Images / SVG / video:**
 - Reference the public R2 URL directly in `<img>` / `<video>` tags (no bundling needed), or
 - Download assets into `src/assets/` or `public/` via a pre-build Node script if you want Astro’s built-in image optimization (`astro:assets`) to process them.
- **Private bucket access:** use AWS SDK v3 (`@aws-sdk/client-s3`) configured with the R2 endpoint and access keys, stored as **Cloudflare Pages environment variables / secrets** — never committed to the repo.

4. Hosting & Deployment: Cloudflare Pages

- Connect the Astro repository to **Cloudflare Pages**.
- Pages runs `astro build` on every trigger and deploys the static output to Cloudflare’s global CDN.
- Generate a **Deploy Hook** under *Project Settings* → *Builds & deployments* → *Deploy Hooks* (per-branch secret URL). Triggering it:


```
curl -X POST "https://api.cloudflare.com/client/v4/pages/webhooks/deploy_hooks/<hook-id>"
```

5. Security Considerations

- Keep R2 write credentials server-side only (Worker secrets or CI/CD secrets); never expose to the browser/client.
- If the bucket must be public for build-time fetches, confirm no sensitive data is stored there.
- Treat the Deploy Hook URL as a credential — anyone with it can trigger a rebuild (low risk, but avoid leaking it publicly).
- Validate uploaded file types/sizes in the update script before writing to R2, to avoid unexpected content reaching the build.
- Use least-privilege R2 API tokens (scoped to the specific bucket, read/write only as needed).

6. Summary Flow

1. R2 bucket holds all assets; updated via S3/R2 API.
2. R2 Event Notification → Queue → Worker (or direct script call) triggers the Cloudflare Pages Deploy Hook.
3. Cloudflare Pages reruns `astro build`, fetching current R2 content.
4. New static site is deployed automatically to Cloudflare's CDN.

7. This repository: Cloudflare deployment and JSON updates

This project is a static Astro site. `src/components/Dogs.astro` reads `public/dogs.json` during the build, and `npm run build` writes the static site to `dist/`. The `package.json` requires Node.js 22.12.0 or newer. The JSON must be present before `astro build` runs because Astro embeds its current data in the generated pages.

JSON in this Astro repository

Connect this Git repository to Cloudflare Pages. Set the production branch to `main`, the build command to `npm run build`, and the build output directory to `dist`. Set `NODE_VERSION` to 22.12.0 or a newer supported Node.js 22 release. Save and deploy. A commit that changes `public/dogs.json` will automatically rebuild and publish the site, so no webhook or Astro Cloudflare adapter is needed for this static site.

Astro's current Cloudflare guide recommends Workers for new projects. Pages is still an option for static output and supplies the Deploy Hooks described here.

JSON in another repository

A webhook starts a deployment; it does not copy data from another repository. The Astro build must separately fetch or copy the updated JSON before running `astro build`. Options include moving the JSON into this repository, downloading it in a pre-build step, or fetching it from R2. For a private source, keep a read-only token in a build secret, not in browser code or Git.

For a separate GitHub repository, create a Pages Deploy Hook in the Cloudflare project's build settings. Store its full URL as the `CLOUDFLARE_DEPLOY_HOOK` secret in the JSON repository. A workflow can trigger the build only when the JSON path changes:

```
name: Rebuild Astro site

on:
  push:
    branches: [main]
    paths:
      - "path/to/dogs.json"

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Trigger Cloudflare Pages deployment
        run: curl --fail --request POST "$CLOUDFLARE_DEPLOY_HOOK"
        env:
          CLOUDFLARE_DEPLOY_HOOK: ${ secrets.CLOUDFLARE_DEPLOY_HOOK }
```

The Pages build hook rebuilds the configured branch. Confirm its pre-build data fetch reads the latest JSON, or the rebuild will publish old content.

Optional R2 event workflow

When several independent clients update objects in R2, use event notifications for relevant object creation/deletion events, deliver them to a Cloudflare Queue, and have a queue consumer Worker call the Deploy Hook:

Uploader -> R2 object change -> R2 event notification -> Cloudflare Queue
-> Queue consumer Worker -> Pages Deploy Hook -> Astro build -> CDN

This decoupled path is useful when uploaders should not manage the hook. For one trusted uploader, calling the hook after a successful upload is simpler.

Build-time JSON and asset access

An Astro build can fetch a public JSON file over HTTPS before generating pages:

```
const response = await fetch("https://assets.example.com/data/dogs.json");
if (!response.ok) throw new Error(`JSON fetch failed: ${response.status}`);
const dogs = await response.json();
```

Reference public R2 custom-domain URLs directly for images and video when appropriate. If Astro must optimize an image at build time, download it before the build into an Astro-processed location. Private R2 access requires server-side credentials stored in Cloudflare or CI secrets and scoped to the required bucket and operations.

Additional security and reliability notes

- Keep deploy-hook URLs and R2 credentials out of Git and browser bundles.
- Use read-only access for build-time fetches when possible; scope write tokens to the required bucket and operations.
- Do not expose a public bucket containing private data.
- Validate uploaded file types and JSON; fail the build if required data cannot be fetched or parsed.
- Filter workflows by the JSON path to avoid deployments for unrelated changes.

8. References

- Astro deployment guide: <https://docs.astro.build/en/guides/deploy/cloudflare/>
- Cloudflare Pages Astro guide: <https://developers.cloudflare.com/pages/framework-guides/deploy-an-astro-site/>
- Cloudflare Pages Deploy Hooks: <https://developers.cloudflare.com/pages/configuration/builds/deploy-hooks/>
- Cloudflare R2 event notifications: <https://developers.cloudflare.com/r2/buckets/event-notifications/>

9. PWA updates `dogs.json` → R2 → webhook → Astro rebuild

This section describes the concrete path for this repository: a PWA edits `dogs.json`, uploads it to Cloudflare R2, and a webhook triggers an Astro static rebuild so the published site matches the new data.

9.1 Current repo behavior (important)

Today `Dogs.astro` and `src/pages/dogs/[slug].astro` read a **local** file:

`public/dogs.json` (read with Node fs during astro build)

A Deploy Hook alone does **not** pull data from R2. The build must either:

1. **Fetch `dogs.json` from R2 at build time** (recommended for PWA/R2), or
2. Keep writing `public/dogs.json` in Git (then a commit rebuilds; no R2 needed).

For the PWA + R2 design, switch the Astro sources to `fetch()` the public R2 URL (or a custom domain on the bucket) before generating pages, for example:

```
const res = await fetch("https://assets.yourdomain.com/data/dogs.json");
if (!res.ok) throw new Error(`dogs.json fetch failed: ${res.status}`);
const dogs = await res.json();
```

Use the same fetch in both the list component and `getStaticPaths()`.

9.2 End-to-end flow

PWA (edit dogs + media)

|
| HTTPS (auth required - never put R2 write keys in the PWA)
v

Upload Worker (or authenticated API)

|
| PutObject: data/dogs.json (+ media/* if needed)
v

Cloudflare R2 bucket

|
+-- Option A: Worker POSTs Deploy Hook after successful upload
+-- Option B: R2 Event Notification → Queue → Worker → Deploy Hook
v

Cloudflare Pages / Workers Builds

|
| npm run build (astro build fetches latest dogs.json from R2)
v

Static site on Cloudflare CDN

9.3 Step-by-step checklist

A. Create the R2 bucket and object layout

1. In Cloudflare Dashboard → **R2** → Create bucket (e.g. `site-content`).
2. Organize prefixes, for example:
 - `data/dogs.json`
 - `media/, icons/, videos/`
3. Expose read access for the **build** (pick one):
 - **Public bucket** or **custom domain** on R2 (simplest for `fetch()`), or
 - Private bucket + S3 credentials as **build secrets** (more secure).
4. Create an R2 API token with least privilege (write for the upload Worker; read-only for the Astro build if using the S3 SDK).

B. Put the Astro repo on Cloudflare (hosting + builds) Cloudflare recommends **Workers** for new Astro projects; **Pages** remains valid for static sites and still provides Deploy Hooks.

Option 1 — Cloudflare Pages (static Astro, Deploy Hooks well documented)

1. Push this repo to GitHub/GitLab.
2. Dashboard → **Workers & Pages** → Create → Import repository.
3. Configure:
 - Production branch: `main`
 - Build command: `npm run build`
 - Build output / directory: `dist`
 - Environment variable: `NODE_VERSION=22.12.0` (or newer Node 22)
4. Save and Deploy. First build should succeed with current `public/dogs.json`.

5. Project Settings → **Builds & deployments** → **Deploy Hooks**:
 - Create hook for `main`, copy the URL.
 - Treat that URL as a secret.

Trigger example:

```
curl -X POST "https://api.cloudflare.com/client/v4/pages/webhooks/deploy_hooks/<HOOK_ID>"
```

Option 2 — Cloudflare Workers + Workers Builds (recommended for new projects)

1. Add a `wrangler.jsonc` (static assets only — no `@astrojs/cloudflare` adapter needed):

```
{
  "name": "majestic-meteorite",
  "compatibility_date": "2026-10-01",
  "assets": {
    "directory": "./dist"
  }
}
```

2. Local smoke test: `npm run build && npx wrangler deploy`
3. Dashboard → Import Git repo → Workers Builds:
 - Build command: `npx astro build` (or `npm run build`)
 - Deploy command: `npx wrangler deploy`
4. Settings → **Builds** → **Deploy Hooks** → create hook for production branch.

Trigger example:

```
curl -X POST "https://api.cloudflare.com/client/v4/workers/builds/deploy_hooks/<HOOK_ID>"
```

C. Wire Astro to read `dogs.json` from R2 at build time

1. Upload a starter `data/dogs.json` to R2 (copy of current `public/dogs.json`).
2. Change build-time reads from `readFileSync('public/dogs.json')` to `fetch(R2_URL)`.
3. Optionally keep a local `public/dogs.json` only for offline `astro dev`, or point `astro.dev` at the same public URL.
4. In Pages/Workers build settings, add any needed env vars (e.g. `DOGS_JSON_URL=https://assets.yourdomain.com/data/dogs.json`).

D. PWA upload path (do not put write credentials in the browser) The PWA must **not** hold R2 write keys. Use a small Cloudflare Worker as the upload gateway:

1. PWA authenticates the user (session cookie, Cloudflare Access, JWT, etc.).
2. PWA PUT/POSTs the new JSON (and optional media) to your Worker.
3. Worker validates JSON schema, size, and auth.
4. Worker writes `data/dogs.json` to R2 via binding or S3 API.
5. Worker (or R2 event pipeline) calls the **Deploy Hook** with POST.

Minimal Worker sketch after a successful R2 write:

```
await env.SITE_BUCKET.put("data/dogs.json", JSON.stringify(dogs), {
  httpMetadata: { contentType: "application/json" },
});
await fetch(env.DEPLOY_HOOK_URL, { method: "POST" });
```

Store `DEPLOY_HOOK_URL` with `wrangler secret put DEPLOY_HOOK_URL`.

E. Alternative: R2 event notifications (no Deploy Hook in the PWA Worker) If several clients write to R2:

1. Enable R2 **Event Notifications** for `PutObject` / `DeleteObject` (optionally filter prefix `data/`).
2. Deliver to a **Cloudflare Queue**.

3. Queue consumer Worker POSTs the Deploy Hook.
4. Useful when uploaders should not know about rebuilds.

F. Verify the loop

1. Edit dogs in the PWA → upload succeeds → object visible in R2.
2. Deploy Hook fires → build appears in Cloudflare deploy history.
3. Build logs show successful fetch of `dogs.json`.
4. Live site list and `/dogs/<slug>/` pages show the new content.

9.4 What goes where

Asset	Store in	Consumed by
<code>dogs.json</code>	R2 <code>data/dogs.json</code>	Astro at build time
Images / SVG / video	R2 <code>media/</code> etc.	Public URLs in HTML, or download into Astro assets if optimizing
Astro source (<code>.astro</code> , CSS)	Git repo	Cloudflare Pages / Workers Builds
Deploy Hook URL	Worker / CI secret	Upload Worker or Queue consumer only
R2 write credentials	Worker secrets	Upload Worker only — never the PWA

9.5 Security reminders for the PWA path

- Never embed R2 write tokens or the Deploy Hook URL in the PWA bundle.
- Authenticate every upload; rate-limit and validate JSON.
- Prefer read-only public URLs (or signed reads) for the Astro build.
- Scope API tokens to one bucket and required operations.
- Deduplicate rebuilds if the PWA saves often (Workers Builds Deploy Hooks can skip redundant queued builds; you can also debounce in the Worker).

9.6 Extra references

- Astro on Cloudflare Workers: <https://developers.cloudflare.com/workers/framework-guides/web-apps/astro/>
- Astro deploy to Cloudflare: <https://docs.astro.build/en/guides/deploy/cloudflare/>
- Workers Builds Deploy Hooks: <https://developers.cloudflare.com/workers/ci-cd/builds/deploy-hooks/>
- Pages Deploy Hooks: <https://developers.cloudflare.com/pages/configuration/builds/deploy-hooks/>
- R2 event notifications: <https://developers.cloudflare.com/r2/buckets/event-notifications/>